



PROGRAMAÇÃO WEB 2 · AULA 04 · GUIA COMPLETO

Primeira página

passo a passo

Do projeto ao trabalho, arquivo por arquivo

Emanuel Nascente · Escola Técnica Mesquita

REACT :D



0 caminho da aula

01

Criar o projeto com o Vite e entender a estrutura de arquivos.

02

Escrever o **Login.jsx** em 12 etapas — uma peça de código por vez.

03

Estilizar no **Login.css** e montar o componente no **App.jsx**.

04

Fazer o trabalho: o botão mostrar / ocultar senha.

Antes de começar

01

Node.js

É o Node que roda o Vite e o npm. Rode os dois comandos abaixo no terminal: se as versões aparecerem, está tudo certo.

```
$ node -v  
$ npm -v
```

Sem resposta? Instale em nodejs.org (versão LTS).

02

VS Code

É onde vamos escrever o código. Instale também a extensão "ES7+ React snippets" — ela ajuda, mas não é obrigatória.

```
$ code .
```

O comando abre a pasta atual dentro do VS Code.

03

Uma pasta só sua

Escolha onde o projeto vai morar e entre nessa pasta pelo terminal antes de criar o projeto. Evite pastas sincronizadas na nuvem.

```
$ cd Documentos/Projetos
```

Caminho sem acento e sem espaço evita dor de cabeça.

Criando o projeto

01

Criar

O Vite monta a pasta do projeto já com o React configurado. Rode o comando e responda às perguntas que ele fizer no terminal.

```
$ npm create vite@latest  
meu-app -- --template react
```

Troque "meu-app" pelo nome que você quiser.

02

Instalar

Entre na pasta que acabou de nascer e instale as dependências. O npm install cria a node_modules — demora um pouco na primeira vez.

```
$ cd meu-app  
$ npm install
```

A node_modules não se edita e não se envia na entrega.

03

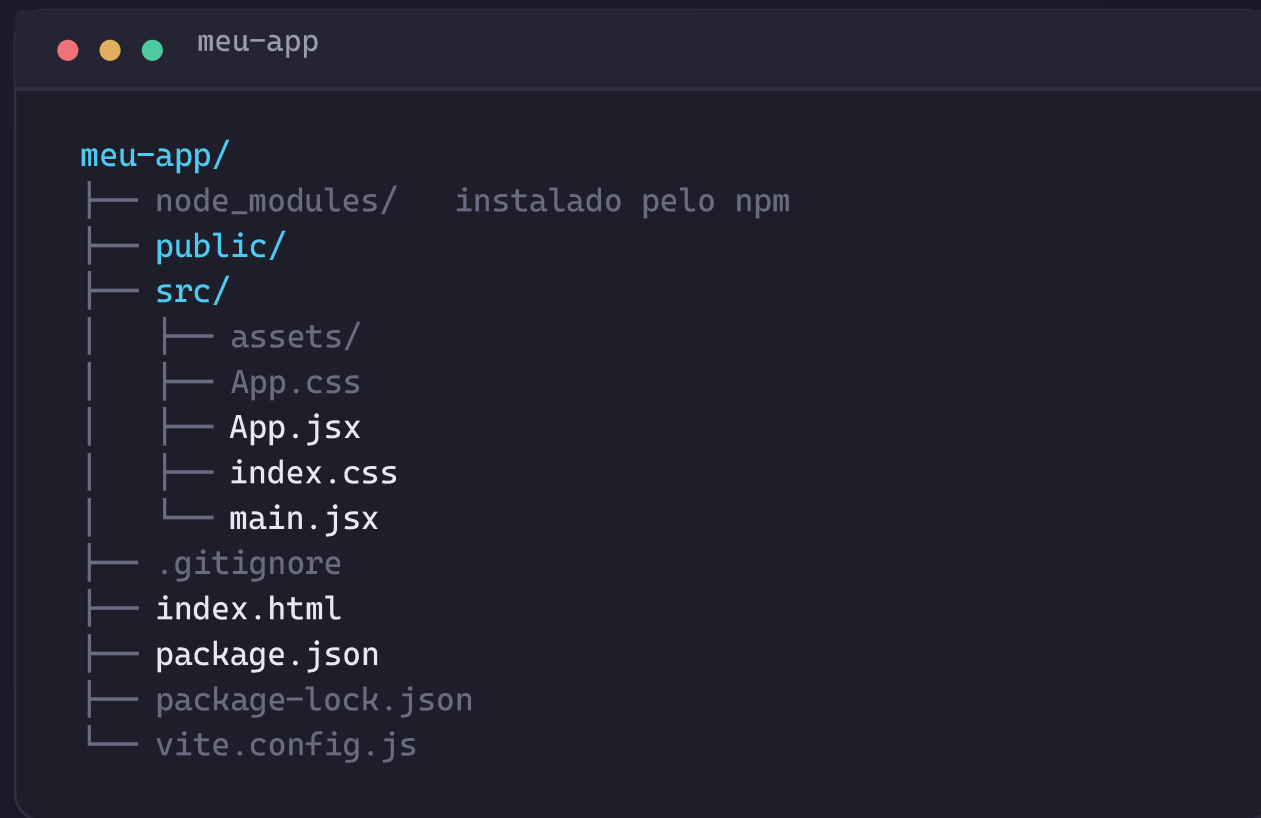
Rodar

Sobe o servidor de desenvolvimento. Deixe rodando: cada vez que você salvar um arquivo, o navegador atualiza sozinho.

```
$ npm run dev
```

Abra o endereço <http://localhost:5173> no navegador.

O que o Vite criou



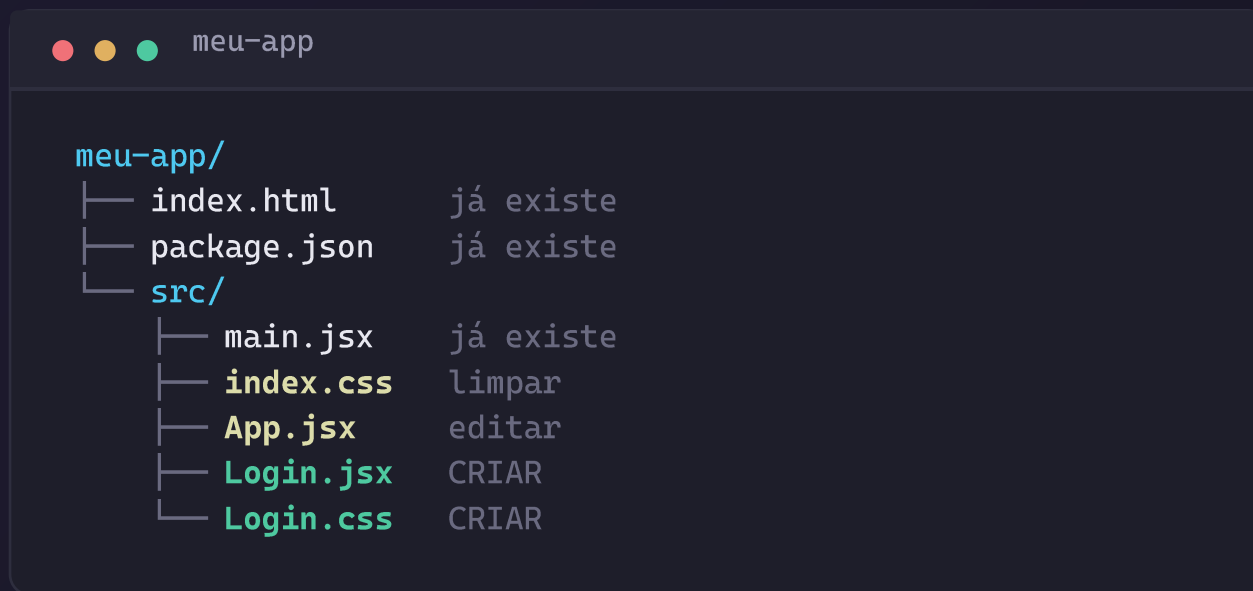
Essa é a pasta recém-criada. Parece muita coisa, mas hoje só quatro arquivos importam.

A **node_modules** guarda as bibliotecas instaladas: nunca se edita nada lá dentro.

Todo o código que a gente escreve vive dentro da pasta **src** — é lá que os nossos arquivos vão nascer.

Os arquivos apagados na lista ao lado são de configuração: deixe como estão.

A estrutura da nossa aula



Ao final da aula a tua pasta src precisa ter exatamente esses cinco arquivos.

Dois deles você vai **criar do zero**: o componente e o estilo dele. Dois já existem e a gente ajusta. Um fica intocado.

Pode apagar o App.css e a pasta assets: nesta aula eles não são usados.

CRIAR arquivo novo, feito por você
editar arquivo que já existe e vamos mexer
cinza não precisa tocar hoje

0 caminho até a tela

01 `index.html` tem uma div vazia com id="root". É a página que o navegador abre.

02 `main.jsx` pega essa div e manda o React desenhar a aplicação dentro dela.

03 `App.jsx` é o componente raiz: ele decide o que aparece na tela.

04 `Login.jsx` é o nosso componente — a tela que vamos construir hoje.

index.html

```
index.html

1  <!doctype html>
2  <html lang="pt-BR">
3    <head>
4      <meta charset="UTF-8" />
5      <title>Login - Aula 04</title>
6    </head>
7    <body>
8      <div id="root"></div>
9      <script type="module"
10        src="/src/main.jsx"></script>
11    </body>
12  </html>
```

Esse arquivo já veio pronto e a gente quase não mexe nele.

A linha em destaque é a única que importa: a `<div id="root">` é o buraco vazio onde o React vai desenhar tudo.

Repare que não existe nenhum texto da tela de login aqui. Toda a página é montada pelo JavaScript.

Dica: mude o `<title>` para o nome da tua aplicação — é o texto da aba do navegador.

main.jsx

```
src/main.jsx

1  import React from "react";
2  import ReactDOM from "react-dom/client";
3  import App from "./App";
4  import "./index.css";
5
6  ReactDOM.createRoot(
7    document.getElementById("root")
8  ).render(
9    <React.StrictMode>
10     <App />
11   </React.StrictMode>
12 );
```

É a ponte entre o HTML e o React. Também já vem pronto.

Ele procura o elemento de id **"root"** e renderiza o **<App />** ali dentro.

A linha **import "./index.css"** é o que faz o estilo global valer para a aplicação inteira.

index.css

src/index.css

```
1  * {  
2    box-sizing: border-box;  
3  }  
4  
5  body {  
6    margin: 0;  
7    font-family: system-ui, sans-serif;  
8  }
```

Apague tudo que veio no index.css e deixe só isso.

O **box-sizing: border-box** faz o padding entrar na conta da largura — sem ele os campos estouram o formulário.

O **margin: 0** no body tira a margem branca que o navegador coloca sozinho.

Trabalhamos em .JSX, não em .JS

Nós trabalhamos no arquivo **.JSX**, não **.JS**.

Trabalhando em JSX nós usamos uma "extensão" do HTML dentro do nosso arquivo JS. Dessa forma temos uma sintaxe um pouco diferente do JavaScript normal.

Na prática: escrevemos marcação parecida com HTML no meio do código JavaScript, e o React se encarrega de transformar isso na tela.

JavaScript

.js

+

HTML

<tags>

=

JSX

.jsx

o esqueleto

```
src/Login.jsx

1  function Login() {
2    return (
3      <h2>Login</h2>
4    );
5  }
6
7  export default Login;
```

Crie o arquivo Login.jsx dentro da pasta src e comece com isso. Salve e a palavra Login já aparece no navegador.

Todo componente React é uma função que devolve marcação. O nome começa com letra MAIÚSCULA — é assim que o React sabe que é um componente, e não uma tag comum.

O **export default** no final libera esse componente para ser usado em outros arquivos.

import

src/Login.jsx

```
1 import { useState } from "react";
2 import "./Login.css";
3
4 function Login() {
5   ...
6 }
7
8 export default Login;
```

O `import { useState } from "react"` vai até a biblioteca do React e traz a função `useState` para dentro do nosso arquivo.

useState é o que chamamos de **Hook**. Ele permite que a função "lembre" de informações entre uma renderização e outra.

A segunda linha importa o CSS. O arquivo ainda não existe — vamos criá-lo no passo 3; deixe a linha comentada até lá se preferir.

Atenção: as chaves em `{ useState }` são obrigatórias.

useState

src/Login.jsx

```
1  function Login() {  
2      const [email, setEmail] = useState("");  
3      const [senha, setSenha] = useState("");  
4      const [erro, setErro] = useState("");  
5  
6      ...  
7  }
```

Cada estado vem sempre em par: o valor e a função que muda esse valor. Os nomes são escolha nossa; `setAlgumaCoisa` é a convenção.

O que está entre parênteses é o **valor inicial**.

Começamos com texto vazio nos três.

Nunca mude o valor na mão (**email = ...**). Só a função `setEmail` avisa o React de que a tela precisa ser redesenhada.

Como o estado funciona

Variáveis comuns não avisam a tela quando mudam. O estado avisa — e é por isso que o React existe. O ciclo é sempre este:

- 01 Você digita no campo e o **onChange** dispara, chamando o setEmail com o texto novo.
- 02 O React guarda o valor e **executa a função Login de novo**, do começo ao fim.
- 03 A tela é redesenhada já com o valor novo. Sem F5, sem document.getElementById, sem innerHTML.

function handleSubmit(e)

```
src/Login.jsx
1    ...
2
3    function handleSubmit(e) {
4        e.preventDefault();
5
6        ...
7    }
8
9    ...
```

Essa é a nossa primeira função aninhada: ela mora dentro da função Login. Estranho para quem vem de outras linguagens, normal em React.

O **e** é o evento — quem disparou, que tipo foi, qual tecla. Poderíamos chamá-lo de evento; e é só convenção.

e.preventDefault() impede o comportamento padrão do formulário, que é recarregar a página inteira. Sem ela, tudo se perde ao clicar em Entrar.

campos vazios

src/Login.jsx

```
1  function handleSubmit(e) {  
2    e.preventDefault();  
3  
4    if (!email || !senha) {  
5      setErro("Preencha todos os campos.");  
6      return;  
7    }  
8  
9    ...  
10 }
```

O **!** significa "não". Então **!email** quer dizer: o e-mail está vazio.

O **||** é o OU: basta um dos dois estar vazio para entrar no if.

O **return** sozinho encerra a função ali mesmo — sem ele, o código seguiria e ainda tentaria conferir a senha.

if / else

```
src/Login.jsx
1      ...
2
3      if (email === "user@email.com" &&
4          senha === "123456") {
5          alert("Login realizado com sucesso!");
6          setErro("");
7      } else {
8          setErro("E-mail ou senha incorretos.");
9      }
10 }
```

Aqui está a conferência: se o e-mail é user@email.com E a senha é 123456, o login deu certo.

O **&&** é o E: as duas condições precisam ser verdadeiras. E **===** compara valor e tipo — em JavaScript use sempre três iguais.

Deu certo: alert na tela e a mensagem de erro é limpa. Deu errado: a mensagem de erro recebe o texto e aparece sozinha na tela.

Isso NÃO é segurança: a senha está escrita no código, que qualquer um lê no navegador. É exercício de useState.

return ()

```
src/Login.jsx
1    ...
2
3    return (
4        <div className="container">
5
6            ...
7
8        </div>
9    );
10 }
```

Fechamos as duas funções e chegamos na parte visual. Daqui para baixo é praticamente HTML.

O return de uma linha só dispensa parênteses; como o nosso ocupa várias linhas, os parênteses são obrigatórios.

Regra de ouro: o return devolve **um único elemento**. Por isso tudo vive dentro da div container.

className, não class: class é palavra reservada do JavaScript, então o JSX usa className.

form / h2

```
src/Login.jsx
1    ...
2
3    <div className="container">
4      <form onSubmit={handleSubmit}>
5        <h2>Login</h2>
6
7      ...
8
9    </form>
10   </div>
```

O **onSubmit={handleSubmit}** liga o formulário à nossa função: quem envia o form chama o handleSubmit.

Repare nas chaves: em JSX, chaves significam "aqui dentro é JavaScript". Sem elas o React entenderia o texto literal.

O h2 é o título do cartão de login — igualzinho ao HTML.

input email

```
src/Login.jsx
1      ...
2
3      <input
4        type="email"
5        placeholder="Email"
6        value={email}
7        onChange={(e) =>
8          setEmail(e.target.value)}
9      />
10
11    ...
```

O `type="email"` avisa o navegador que o campo é um e-mail — no celular o teclado já abre com o @.

O `value={email}` e o `onChange` juntos fazem o campo virar um componente controlado: o React passa a ser o dono do que está escrito.

A tag fecha em `/>`. Em JSX toda tag precisa ser fechada, inclusive `input`, `img` e `br`.

Atenção: `value` **sem** `onChange` trava o campo — você digita e nada aparece.

0 campo controlado, em 3 tempos

Essa ida e volta acontece a cada tecla digitada. Entender esse circuito é entender React inteiro:

01 A tecla é digitada → o **onChange** recebe o evento e lê o **e.target.value**.

02 O **setEmail** guarda esse texto no estado e o React redesenha o componente.

03 O campo é redesenhado com **value={email}** — a letra só aparece porque o estado mudou.

input senha

```
src/Login.jsx
1      ...
2
3      <input
4        type="password"
5        placeholder="Senha"
6        value={senha}
7        onChange={(e) =>
8          setSenha(e.target.value)}
9      />
10
11    ...
```

É o mesmo campo de antes, trocando três coisas: o type, o placeholder e o estado usado.

O **type="password"** é o que troca as letras por bolinhas na tela.

Guarde bem esta linha: é exatamente ela que o trabalho da aula pede para deixar dinâmica.

{erro && ...}

```
src/Login.jsx
1      ...
2
3      {erro && <p className="erro">{erro}</p>}
4
5      ...
```

Lendo em voz alta:

se a variável **erro** tiver algum texto, ENTÃO mostre o parágrafo **<p>** com a mensagem.

É o curto-circuito do JavaScript: com o erro vazio, o React simplesmente não desenha nada — nenhum espaço em branco fica sobrando na tela.

button

```
src/Login.jsx
1      ...
2
3      <button type="submit">Entrar</button>
4    </form>
5  </div>
6  );
7  }
8
9  export default Login;
```

Como o botão é **type="submit"** e está dentro do form, clicar nele dispara o `onSubmit` — ou seja, o nosso `handleSubmit`.

Pressionar Enter dentro de um campo faz exatamente a mesma coisa. Isso é de graça: veio do form.

Confira o fechamento na ordem certa: form, div, o parêntese do return e a chave da função.

export default Login

Sem essa linha o `App.jsx` não consegue importar o componente.

Login.jsx (1 de 2)

src/Login.jsx

```
1  import { useState } from "react";
2  import "./Login.css";
3
4  function Login() {
5    const [email, setEmail] = useState("");
6    const [senha, setSenha] = useState("");
7    const [erro, setErro] = useState("");
8
9    function handleSubmit(e) {
10      e.preventDefault();
11
12      if (!email || !senha) {
13        setErro("Preencha todos os campos.");
14        return;
```

src/Login.jsx

```
15    }
16
17    if (email === "user@email.com" &&
18        senha === "123456") {
19      alert("Login realizado com sucesso!");
20      setErro("");
21    } else {
22      setErro("E-mail ou senha incorretos.");
23    }
24  }
25
26  return (
27    <div className="container">
28      <form onSubmit={handleSubmit}>
```

Confira o teu arquivo linha a linha: **a lógica toda cabe nas primeiras 28 linhas.**

Login.jsx (2 de 2)

src/Login.jsx

```
29      <h2>Login</h2>
30
31      <input
32        type="email"
33        placeholder="Email"
34        value={email}
35        onChange={(e) =>
36          setEmail(e.target.value)}
37      />
38
39      <input
40        type="password"
41        placeholder="Senha"
42        value={senha}
```

src/Login.jsx

```
43        onChange={(e) =>
44          setSenha(e.target.value)}
45      />
46
47      {erro && <p className="erro">{erro}</p>}
48
49      <button type="submit">Entrar</button>
50    </form>
51  </div>
52  );
53  }
54
55  export default Login;
```

Da linha 26 em diante é a parte visual: **um container, um form e quatro filhos dentro dele.**

Criando o Login.css

A página deve ser estilizada de acordo com as tuas preferências. Vamos usar aqui a estética da aula — e para isso o passo a passo é:

A Criar um arquivo novo dentro da pasta **src**, ao lado do Login.jsx.

B Nomear ele **Login.css** — com L maiúsculo, igual ao componente.

C Importar no topo do Login.jsx com **import './Login.css';**

Login.css · a moldura

src/Login.css

```
1  .container {
2    display: flex;
3    justify-content: center;
4    align-items: center;
5    height: 100vh;
6    background: #f0f2f5;
7  }
8
9  form {
```

src/Login.css

```
10    background: white;
11    padding: 2rem;
12    border-radius: 8px;
13    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
14    display: flex;
15    flex-direction: column;
16    gap: 1rem;
17    width: 300px;
18  }
```

O container ocupa a altura toda da tela (**100vh**) e centraliza o formulário com flex. O form é o cartão branco.

Login.css • os campos

src/Login.css

```
19
20  input {
21    padding: 0.75rem;
22    border: 1px solid #ccc;
23    border-radius: 6px;
24    font-size: 1rem;
25  }
26
27  button {
28    padding: 0.75rem;
29    background: #4f46e5;
30    color: white;
31    border: none;
32    border-radius: 6px;
```

src/Login.css

```
33    font-size: 1rem;
34    cursor: pointer;
35  }
36
37  button:hover {
38    background: #4338ca;
39  }
40
41  .erro {
42    color: red;
43    font-size: 0.85rem;
44    margin: 0;
45  }
```

Sem seletor de classe: **input** e **button** pegam todos os campos e botões da página — dá para começar assim.

App.jsx

```
src/App.jsx

1  import Login from "./Login";
2
3  function App() {
4    return <Login />;
5  }
6
7  export default App;
```

Apague o que o Vite deixou no App.jsx e deixe só isso.

O `<Login />` é o nosso componente usado como se fosse uma tag do HTML. Isso só é possível por causa do export default lá no fim do Login.jsx.

Salve tudo, volte ao navegador: a tela de login já deve estar lá, centralizada.

Não esqueça: o caminho é `"./Login"` — com ponto e barra, sem a extensão.

Testando a tua tela

01

Clique em Entrar com os campos vazios: a mensagem "Preencha todos os campos." precisa aparecer em vermelho.

02

Digite qualquer e-mail e senha errados: a mensagem tem que mudar para "E-mail ou senha incorretos.".

03

Digite **user@email.com** e **123456**: o alerta de sucesso aparece e o erro some.

04

Aperte Enter dentro de um campo: tem que funcionar igual ao clique no botão.

Os erros mais comuns

01

Tela branca

Quase sempre é erro de digitação no JSX. Abra o console do navegador (F12) e leia a primeira linha vermelha: ela diz o arquivo e a linha.

```
$ F12 → Console
```

Tag não fechada e chave a mais são os campeões.

02

Digito e não aparece

O campo tem value mas está sem o onChange. O React manda o valor do estado para a tela e nada devolve o que você digitou.

```
$ value + onChange
```

Os dois sempre andam juntos num campo controlado.

03

A página recarrega

Faltou o e.preventDefault() na primeira linha do handleSubmit. O formulário fez o que ele faz desde sempre: recarregar tudo.

```
$ e.preventDefault();
```

Sinal clássico: a tela pisca e o campo esvazia.

Mais coisas para conferir

- 01 Escreveu `class` no lugar de `className` — o estilo não aplica e o console avisa.
- 02 Nome do arquivo com outra letra: `login.jsx` e `Login.jsx` são arquivos diferentes para o import.
- 03 Esqueceu de salvar o arquivo. O Vite só atualiza a tela depois do Ctrl+S.
- 04 O terminal parou: o `npm run dev` precisa ficar rodando o tempo todo enquanto você trabalha.

Mostrar / ocultar senha

A entrega da aula é um botãozinho dentro do campo de senha que alterna entre esconder e revelar o que foi digitado. São três mudanças no Login.jsx:

01 Um estado novo: `verSenha`, que começa em `false`.

02 O `type` do campo deixa de ser fixo e passa a depender desse estado.

03 Um botão ao lado do campo que inverte o estado a cada clique.

verSenha

src/Login.jsx

```
1    ...
2    const [erro, setErro] = useState("");
3    // guarda se a senha está visível
4    const [verSenha, setVerSenha] =
5      useState(false);
6
7    ...
```

Mais um estado, junto dos outros três. A diferença é o valor inicial: em vez de texto vazio, o booleano `false`.

false significa "a senha começa escondida" — que é o comportamento que a gente quer ao abrir a tela.

Nome em português e sem abreviação: `verSenha` diz o que guarda. Isso conta na correção.

type dinâmico

```
src/Login.jsx
1      ...
2
3      <input
4      type={verSenha ? "text" : "password"}
5      placeholder="Senha"
6      value={senha}
7      onChange={(e) =>
8          setSenha(e.target.value)}
9      />
10
11      ...
```

Antes o type era o texto fixo **"password"**. Agora ele é JavaScript — e por isso vai entre chaves.

Isso é o **operador ternário**: condição ? valor se verdadeiro : valor se falso.

Leia assim: se verSenha for verdadeiro use text (aparece), senão use password (bolinhas).

É uma linha só: o ternário substitui um if inteiro dentro do JSX, onde if não é permitido.

o botão

```
src/Login.jsx
1      ...
2
3      <button
4        type="button"
5        className="ver-senha"
6        onClick={() => setVerSenha(!verSenha)}
7      >
8        {verSenha ? "Ocultar" : "Mostrar"}
9      </button>
10
11    ...
```

type="button" é obrigatório: sem isso o botão viraria submit e tentaria fazer login a cada clique.

O **!verSenha** é o inverso do valor atual — é o que faz um clique alternar entre os dois estados.

O texto do botão também é um ternário: ele muda junto, de Mostrar para Ocultar.

o campo pronto

```
src/Login.jsx

1  <div className="campo-senha">
2    <input
3      type={verSenha ? "text" : "password"}
4      placeholder="Senha"
5      value={senha}
6      onChange={e => setSenha(e.target.value)}
7    />
8
9    <button type="button" className="ver-senha"
10      onClick={() => setVerSenha(!verSenha)}>
11      {verSenha ? "Ocultar" : "Mostrar"}
12    </button>
13  </div>
```

O input e o botão agora moram juntos dentro de uma div com a classe campo-senha. É ela que permite encostar o botão dentro do campo pelo CSS.

Cuidado ao mover o input para dentro da div: a indentação muda, mas o conteúdo é o mesmo de antes.

Esse bloco inteiro entra no lugar do input de senha da etapa 10.

Login.css • o botão

src/Login.css

```
46  /* botão mostrar / ocultar senha */
47
48  .campo-senha {
49      position: relative;
50      display: flex;
51  }
52
53  .campo-senha input {
54      width: 100%;
55      padding-right: 5rem;
56  }
57
58  .ver-senha {
59      position: absolute;
60      right: 0.4rem;
61      top: 50%;
```

src/Login.css

```
62      transform: translateY(-50%);
63      background: transparent;
64      color: #4f46e5;
65      border: none;
66      font-size: 0.8rem;
67      padding: 0.3rem 0.5rem;
68      cursor: pointer;
69      width: auto;
70  }
71
72  .ver-senha:hover {
73      background: #eef0fe;
74      border-radius: 4px;
75  }
```

Acrescente no final do Login.css. **position: absolute** tira o botão do fluxo e o encaixa dentro do campo; o padding-right abre espaço para ele.

Antes de mandar o trabalho

Envie a pasta do projeto SEM a node_modules — ela é recriada com um npm install. E lembre do aviso da aula:



Conferir e-mail e senha no front-end é só exercício. Numa aplicação real, qualquer pessoa lê essa senha no navegador.



A autenticação de verdade acontece no servidor — é o que vamos construir no back-end em Python (FastAPI) do ponto eletrônico.



Próxima aula: começamos a nova página da aplicação, conforme conversaremos em sala :)